A High-Performance Vectorized Framework for Multiobjective Evolutionary Optimization in Python

Nicolás R. Uribe, Alberto Herrán, Antonio J. Nebro, Javier Del Ser, and J. Manuel Colmenar

***Abstract*—Despite the interpreted nature of Python, which makes metaheuristics implemented in this language inherently slower than those of frameworks written in compiled languages, it remains a widely adopted platform for empirical studies in multi-objective optimization. Most existing Python libraries further aggravate this limitation by managing populations as collections of individual solution objects, preventing effective vectorization and keeping hot-path computations in the interpreter. In this paper, we introduce VAMOS (Vectorized Architecture for Multiobjective Optimization Studies), a Python framework that stores the population state in dense numerical arrays dispatching hot-path computations to pluggable backends and keeping the algorithmic logic independent of the numerical substrate. VAMOS implements eight multi-objective evolutionary algorithms (MOEAs) covering dominance, decomposition, and indicator-based paradigms, with a richer component-level configurability than its competitors. It also provides an automatic algorithm configuration tool, together with a browser-based graphical interface to enhance accessibility for non-expert users. The performance of VAMOS is compared to the existing Python frameworks for multi-objective optimization on several benchmark suites. Paired Wilcoxon signed-rank tests with Holm correction confirm that VAMOS achieves statistically equivalent solution quality to state-of-the-art frameworks, with a reduction in wall-clock runtime.**

***Index Terms*—Evolutionary algorithms, Multiobjective optimization, Software framework, Vectorization**

I. Introduction

MULTI-OBJECTIVE optimization problems (MOPs) arise whenever trade-offs exist between conflicting objectives, requiring the computation of an accurate approximation of the Pareto-optimal set rather than a single optimum [1]. Such problems are common in engineering design, machine learning, scheduling, and resource allocation, among other domains. In this field, multi-objective evolutionary algorithms (MOEAs) remain a dominant approach due to their robustness, population-based anytime behavior, and ability to return a set of non-dominated solutions in a single run [2].

The widespread adoption of MOEAs has been fueled in large part by the availability of software frameworks that lower the implementation barrier and facilitate reproducible experimentation. Early efforts such as *PISA* [3] established the concept of modular, platform-independent interfaces for multi-objective search. The *jMetal* framework [4] later consolidated a comprehensive Java-based toolkit that became a reference for algorithm development and research studies. More recently, *PlatEMO* [5] has provided a MATLAB platform with a large catalog of algorithms and test problems. In parallel, the Python ecosystem has seen the emergence of dedicated libraries, such as *pymoo* [6], *jMetalPy* [7], Platypus [8], and *DEAP* [9], that leverage Python's accessibility and rich scientific stack.

As Python has become a widely adopted platform for MOEA research and experimentation, three practical limitations hinder its broader adoption. First, the interpreted nature of Python, combined with libraries that keep hot-path computations inside the interpreter [7]–[9], makes the inner loops of pure-Python implementations inherently slower than those of frameworks written in compiled languages such as C++ or Java. Second, existing MOEA frameworks often provide limited configurability at the component level and do not allow to attach external archives, useful to allow NSGA-II to handle problems with more than two objectives [10], with the exception of *jMetal* [4]. Third, most existing frameworks present a high entry barrier for non-expert users while simultaneously lacking the necessary flexibility for advanced researchers to achieve fine-grained algorithmic configuration. To overcome these limitations, we introduce VAMOS, a Python-based package designed with a single guiding principle: keep algorithm state in dense arrays and push hot loops into vectorized kernels. Specifically, the main contributions of VAMOS are summarized as follows:

a) *Performance*: A unified array-based internal representation with pluggable compute kernels that reduces inner-loop overhead relative to object-centric libraries.
b) *Configurability*: Eight MOEA implementations with high component-level configurability and an auto tune tool.
c) *Accessibility*: An accessible API and tooling for novice-to-expert workflows.

The rest of the paper is organized as follows. Section II reviews the existing Python MOEA frameworks. Section III presents the VAMOS framework. Sections V, VI, and VII address the three contributions—performance, configurability, and accessibility—respectively. Finally, Section VIII draws conclusions and outlines directions for future work.

II. Related Work

In this section, we review the most relevant Python libraries for MOEAs, namely *pymoo*, *jMetalPy*, *DEAP*, *Platypus*, and *PyGMO*. Table I summarizes the main differences between

N. R. Uribe, A. Herrán, and J. M. Colmenar are with the Department of Computer Sciences, Universidad Rey Juan Carlos, Móstoles, 28933 Madrid, Spain (e-mail: nicolas.rodriguez@urjc.es; alberto.herran@urjc.es; josemanuel.colmenar@urjc.es).

A. J. Nebro is with the Department of Lenguajes y Ciencias de la Computación, ITIS Software, University of Málaga, 29071 Málaga, Spain (e-mail: ajnebro@uma.es).

J. Del Ser is with TECNALIA, Basque Research & Technology Alliance (BRTA), Derio, 48160, Spain, and also with the University of the Basque Country (UPV/EHU), Leioa, 48940, Spain (e-mail: javier.delser@tecnalia.com).

Corresponding author: Alberto Herrán (alberto.herran@urjc.es).

TABLE I
COMPARISON OF PYTHON MOP FRAMEWORKS. ✓ FULL SUPPORT ❍ PARTIAL/LIMITED SUPPORT ✗ ABSENCE

Feature	pymoo	jMetalPy	DEAP	Platypus	PyGMO	VAMOS
Performance indicators	✓	❍	✗	❍	✗	✓
Dense array model	❍	✗	✗	✗	✗	✓
Pluggable backends	✗	✗	✗	✗	✗	✓
Parallelism	❍	❍	✓	❍	✓	❍
Dominance-based	✓	✓	❍	✓	❍	✓
Decomposition-based	✓	❍	✗	✓	❍	✓
Indicator-based	❍	❍	✗	❍	✗	✓
Interchangeable operators	❍	❍	✓	❍	✗	✓
Conf. steady-state mode	✓	❍	❍	✗	✗	✓
External archive	✗	✗	❍	❍	✗	✓
Automatic algorithm selection	✗	✗	✗	✗	✗	✓
Min. lines of code	∼10	∼15	∼25	∼8	∼5	2
Graphical user interface	✗	✗	✗	✗	✗	✓
Active maintenance	✓	❍	❍	❍	✓	✓

Python multi-objective optimization frameworks relevant to this work, including all the features required for a high performance, configurable and accessible framework. In this table, features 1–4 evaluate performance, 5–9 assess configurability, and 10–12 highlight accessibility. VAMOS is the only framework that combines a dense array population model, pluggable computation backends, rich component-level configurability, a minimal two-line entry point (measured as lines of code needed for a minimal NSGA-II run on a built-in benchmark) and a graphical user interface.

pymoo [6] is a comprehensive and widely used framework offering diverse MOEAs and utilities. While its object-oriented architecture via subclassing promotes customization, it tightly couples algorithmic logic to internal data structures, complicating the systematic vectorization of computations.

jMetalPy [7] adapts the Java *jMetal* [4] framework. It provides robust component-based extensibility and runtime monitoring. However, its reliance on collections of individual solution objects prioritizes design clarity over performance, precluding backend-level array vectorization.

DEAP [9] provides a generic, highly flexible functional toolkit for prototyping evolutionary algorithms. While powerful, compiling sophisticated multi-objective pipelines requires substantial user effort. Furthermore, its per-individual object model shares the vectorization limits of *jMetalPy*.

Platypus [8] is a lightweight, beginner-friendly MOEA library. However, its development has stalled, and it relies on object-centric representations similar to *DEAP* and *jMetalPy*.

PyGMO [11] wraps the C++ *pagmo2* framework, prioritizing performance and parallel optimization (island model) via compiled extensions. However, its architecture favors continuous single-objective tasks. Its multi-objective capabilities restrict component-level granular tuning and tightly bundle archiving strategies, limiting its utility for algorithm design.

Distinct from these CPU-based Python libraries, recent specialized solutions target divergent architectures and problem scopes. For example, *EvoX* [12] provides a functional programming model specifically tailored for mapping evolutionary workloads onto distributed GPU resources. Other contributions focus on specialized settings, such as coevolutionary constrained optimization [13], bi-knowledge transfer for multimodal problems [14], mixed-integer linear pipelines [15], and LLM-assisted code generation [16].

Overall, while existing libraries span a wide spectrum between usability and performance, none explicitly decouple algorithmic logic from the numerical substrate to enable transparent CPU backend substitution. Furthermore, they either target specific niche scenarios or fail to combine a low entry barrier for domain experts with fine-grained configurability for researchers. This gap motivates VAMOS, which provides a vectorized, backend-agnostic architecture aimed specifically at MOEA studies and reproducible cross-framework benchmarking on standard hardware.

III. VAMOS FRAMEWORK

The design of VAMOS is motivated by three practical gaps identified in existing Python MOEA libraries: *performance*, *configurability*, and *accessibility*.

a) Performance: As the population state remains in dense arrays throughout the optimization loop, hot-path computations are dispatched to vectorized or compiled kernels. As we will see in Section V, this design yields substantial runtime reductions across multiple MOEA variants and benchmark families without degrading solution quality. Moreover, switching the compute backend only requires a single-parameter change transparent to algorithm code. No other Python MOEA framework allows swapping the entire computational substrate, from array operations to compiled kernels, without modifying the algorithm implementation. Similarly, evaluation strategies (serial, multiprocessing or distributed) are selected via configuration rather than code changes.

b) Configurability: Most frameworks provide a small set of high-level parameters (population size, crossover/mutation probabilities), but do not expose fine-grained control over offspring batch size, archive attachment, or operator adaptation. VAMOS exposes all of these as first-class configuration options through a builder API. Moreover, VAMOS includes a tuning module exposed both programmatically and via the CLI. It supports random search and an irace-inspired racing procedure [17] with multi-fidelity evaluation, warm-starting and parallel execution. The same interface can also delegate to model-based backends such as Optuna [18], SMAC3 [19], and BOHB [20], which are provided as optional dependencies.

c) Accessibility: Most frameworks require importing separate modules for the algorithm selection, problem definition, used operators, and termination criteria, then wiring them together. This level of boilerplate assumes familiarity with MOEA terminology and framework implementation details, which is a barrier for domain experts who need to solve MOPs but lack a background in metaheuristics. VAMOS addresses this gap at multiple levels (see Section VII): 1) two lines of code are sufficient to run a complete optimization with sensible defaults; 2) a problem factory turns any Python callable into a problem object without subclassing, and an interactive CLI wizard provides a ready-to-run script from a guided questionnaire; 3) VAMOS provides automatic algorithm selection that maps problem traits to a suitable default algorithm. Additionally, when finer control is needed, the builder API ensures that the framework scales from novice to expert use without a different API.

IV. VAMOS Architecture

The core architecture of VAMOS is organized into four layers, as shown in Fig. 1: a) the *Foundation Layer* providing compute kernels, problem definitions, quality indicators, encodings, constraints, and evaluation backends; b) the *Engine Layer* implementing MOEAs on top of shared components, variation operators, external archives, and a tuning subsystem; c) the *Experiment Layer* exposing the unified `optimize()` entry point, CLI tools, benchmarking utilities, and results analysis; and d) the *UX Layer* providing VAMOS Studio (a Streamlit-based interactive dashboard), a visual Problem Builder, a Results Explorer, and an onboarding wizard.

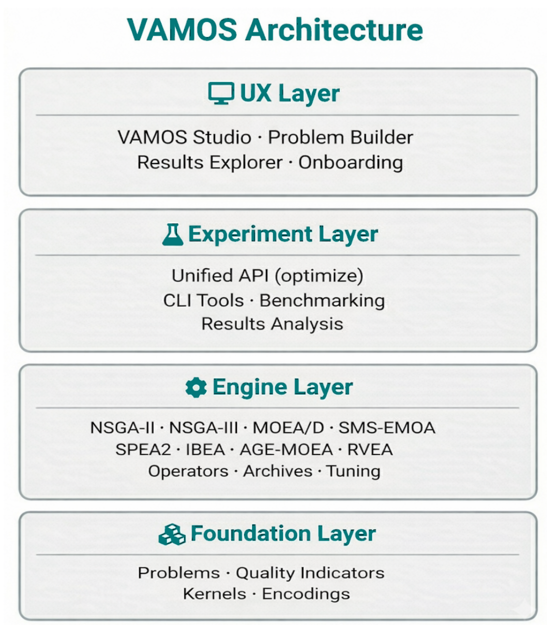

Fig. 1. VAMOS four-layer architecture.

A. Foundation Layer

Given a MOP with n decision variables and m objectives, the core design choice in VAMOS is to represent a population of size N as a pair of dense arrays, one for decision variables $X \in \mathbb{R}^{N\times n}$ and another for objective values $F \in \mathbb{R}^{N\times m}$. In this way, algorithmic logic is separated from numerical kernels that implement the corresponding core computations. Specifically, VAMOS provides three backends to avoid operate over per-individual Python objects, that can be selected through a single configuration parameter and are transparent to algorithm code:

1) *NumPy* as a baseline backend enabling vectorized array operations [21].
2) *Numba* as a just-in-time (JIT) compilation of hot operators and utilities, reducing Python overhead [22].
3) *MooCore* as an optional C-backed kernels for Pareto ranking and hypervolume-based utilities [23].

In addition to accelerating arithmetic kernels, VAMOS reduces overhead by minimizing Python-side allocations which allows variation operators to reuse pre-allocated workspaces and algorithms maintain state as contiguous arrays. This design is particularly beneficial in inner-loop routines such as tournament selection, non-dominated sorting, crowding-distance computation, and archive updates.

Fig. 2 provides an intuitive view of how these frameworks represent and manipulate a population. All frameworks implement the same selection/variation/survival loop, but differ in internal data model and hot-loop granularity. In VAMOS (left) the population is stored in dense arrays (X, F) and the selection/variation/survival operations are applied via array-oriented kernels (NumPy/Numba/MooCore). In contrast, *pymoo* (center) and *jMetalPy* (right) typically manage the population as a collection of solution objects. Specifically, *pymoo* uses a population container of individual objects backed by NumPy arrays, mixing array computation with object overhead, and *jMetalPy* manages a list of solution objects with per-solution method calls.

Beyond kernels, the Foundation Layer hosts the problem abstraction, a base `Problem` class that declares the number of decision variables, objectives, and optional constraints. A problem registry enables lookup by name (e.g., `"zdt1"`, `"dtlz2"`), and a `make_problem` factory converts any Python callable into a problem object without subclassing. Built-in benchmark families include ZDT [24], DTLZ [25], WFG [26], ZCAT [27], CEC 2009 [28], and the real-world problems collections RE [29] and RWA [30]. The layer also provides five variable encodings (real, binary, integer, permutation, and mixed), a constraint-handling module with a declarative DSL, quality indicators (hypervolume, generational distance, inverted generational distance, and spread), and pluggable evaluation backends (serial, multiprocessing, and distributed via Dask) that are selected through configuration rather than code changes.

B. Engine Layer

VAMOS implements the eight MOEAs shown in Table II, covering the three major paradigms: dominance, decomposition, and indicator-based. All eight algorithms are built on top of the shared selection/variation/survival components ensuring consistent operator semantics and enabling component-level substitution without algorithm-specific code changes, so that practitioners can compare paradigms under identical infrastructure and configuration conventions. In addition to the algorithm's selection paradigm, Table II summarizes the supported encodings, number of available crossover and mutation operators, algorithm-specific parameters, and the total number of configurable parameters (base + conditional) in the tuning configuration space. The parameter counts aggregate operators across all supported encodings; in any single configuration with a fixed encoding, between 11 and 18 parameters are

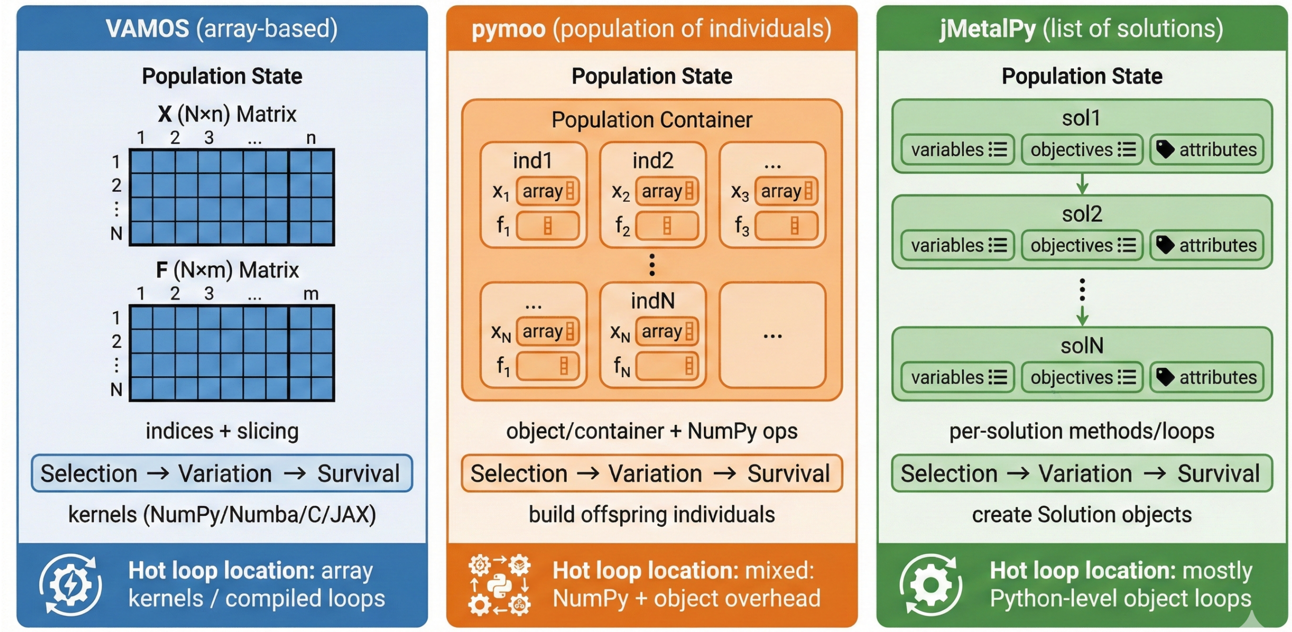

Fig. 2. Comparing population representation across Python MOEA frameworks.

TABLE II
OVERVIEW OF THE EIGHT MOEAS IMPLEMENTED IN VAMOS AND THEIR CONFIGURABLE PARAMETER SPACES ACROSS ALL SUPPORTED ENCODINGS. R = REAL, P = PERMUTATION, B = BINARY, I = INTEGER, M = MIXED

Algorithm	Ref.	Paradigm	Encodings	#Cx	#Mx	Specific Parameters	#Params
NSGA-II	[31]	Dominance	R, P, B, I, M	25	24	Offspring ratio, repair, external archive (capacity factor, pruning)	35
NSGA-III	[32]	Dominance	R, P, B, I, M	21	22	Reference directions (auto-computed)	22
SPEA2	[33]	Dominance	R, P, B, I, M	21	22	Archive size, k-nearest neighbors	24
AGE-MOEA	[34]	Dominance	R, P, B, I, M	21	22	Archive policies (crowding, HV, ε-grid, hybrid)	26
MOEA/D	[35]	Decomposition	R, P, B, I, M	17	20	Aggregation (Tchebycheff, WS, PBI), T, δ, η	18
RVEA	[36]	Decomposition	R, P, B, I, M	21	22	Partitions, α decay, adaptation frequency	28
SMS-EMOA	[37]	Indicator	R, P, B, I, M	21	22	Reference point (adaptive)	22
IBEA	[38]	Indicator	R, P, B, I, M	21	22	Indicator type, scaling factor κ	24

simultaneously active. The parameter counts reflect the tuning configuration spaces used by the built-in autotuner (up to 35 for NSGA-II), illustrating the breadth of component-level configurability that VAMOS exposes as first-class configuration options.

Algorithms in VAMOS expose a uniform `run(problem, termination, seed, eval_backend, live_viz)` interface, which the high-level `optimize()` entry point delegates to after resolving defaults and configuring operators. Several algorithms also support an *ask–tell* loop for streaming or interactive use. Objective evaluations are dispatched through a pluggable evaluation-backend protocol, enabling serial evaluation for controlled benchmarking and parallel evaluation (multiprocessing or distributed) when objective functions are expensive. Because evaluation is decoupled from algorithmic state, the algorithm core remains deterministic under a fixed random seed regardless of whether evaluations run serially or in parallel.

VAMOS supports real-valued, binary, integer, permutation, and mixed-variable encodings through a variation pipeline that composes selection, crossover, mutation, and optional repair operators. For continuous domains, the default configuration uses Simulated Binary Crossover (SBX) and Polynomial Mutation (PM), with mutation probability typically set to $1/n$ where n is the number of decision variables. Binary and integer encodings use standard crossover (one-point, two-point, or uniform) and bit-flip or bounded mutation, while permutation encodings provide order and cycle crossover with insertion mutation. Mixed-variable problems combine encoding-specific operators transparently. Encodings are normalized internally to ensure consistent operator resolution and avoid silent configuration mismatches.

The engine layer also provides optional external archives, both bounded (with crowding-distance or hypervolume-based pruning) and unbounded, that accumulate non-dominated solutions independently of the evolving population. This decouples the population-based search from the elite set, which is particularly useful for problems with more than two objectives [10]. Common quality indicators are provided by the foundation layer, including hypervolume, generational distance (GD), inverted generational distance (IGD), and spread. Hypervolume computation can leverage the MooCore C backend when available (MooCore is an optional dependency because it requires C compilation, which may not be available on all platforms), with pure-Python fallback implementations for low-dimensional cases.

C. Experiment Layer

The experiment layer hosts the unified `optimize()` entry point that resolves defaults, instantiates the algorithm and problem, and delegates to the engine. It also provides a command-line interface with subcommands for running optimizations, hyperparameter tuning, ablation studies, diagnostics, and launching VAMOS Studio. A benchmark runner executes standardized multi-seed studies under a shared protocol (fixed problem definitions, evaluation budgets, and seeds) and emits machine-readable artifacts (per-seed objective values, summary CSVs, and LaTeX-ready tables), reducing transcription errors and enabling end-to-end reproducibility.

D. UX Layer

The UX layer provides *VAMOS Studio*, a Streamlit-based [39] dashboard designed to make multi-objective optimization accessible to users with no programming experience. It offers three main views: (i) a *Welcome* tab with a first-launch onboarding wizard and quick-reference tables for the CLI and Python API; (ii) a *Problem Builder* that lets users write objective and constraint functions in the browser, adjust bounds and algorithm settings, and see the Pareto front update live after each preview run; and (iii) an *Explore Results* tab for loading completed studies, comparing algorithms, and ranking solutions with multi-criteria decision-making methods. The *Problem Builder* ships with domain-specific templates (engineering design, machine learning, and scheduling) so that domain experts can start from a realistic example rather than a blank editor. Generated problems can be exported as standalone Python scripts or run directly through the API. In addition to Studio, the UX layer includes analysis utilities for statistical comparison of algorithm runs (Wilcoxon tests, effect sizes), visualization helpers for Pareto fronts and convergence plots, and multi-criteria decision-making scorers (TOPSIS, weighted sums).

V. Performance

This section describes the experimental setup used to evaluate VAMOS, which is centered on assessing its computational performance compared with other frameworks. It is worth mentioning that we will focus on continuous optimization problems, as analyzing other encodings is out of the scope of this paper because of space constraints. We define the benchmark suite, justify the choice of compared frameworks and algorithms, describe the cross-framework semantic-alignment protocol, and specify the runtime measurement and quality-indicator protocols.

A. Compared Frameworks and Algorithms

Although VAMOS implements eight MOEAs, our cross-framework comparison focuses on those with the closest semantic matches across libraries: NSGA-II, SMS-EMOA, and MOEA/D. This scope avoids confounding effects from missing implementations or materially different operator and termination semantics in specific frameworks. Furthermore, these algorithms are the most representative of the three commonly accepted categories of MOEAs: dominance-based (NSGA-II), indicator-based (SMS-EMOA), and decomposition-based (MOEA/D).

In the case of NSGA-II, we compare three variants: the baseline (i.e., standard generational NSGA-II), a steady-state version, and an unbounded archive-based version. NSGA-II is a generational genetic algorithm, but its working principles are preserved in a steady-state version (i.e., the offspring population contains only one solution), and some studies [40] have shown that this variant produces fronts with improved diversity compared to the standard version. NSGA-II has traditionally been considered a poorly performing algorithm for problems with more than two objectives, but it was reported in [10] that incorporating an unbounded external archive improves the performance of NSGA-II on three-objective problems. Both variants enhance the base NSGA-II but at the cost of increased computational overhead.

Our choice of algorithms restricts the comparison of VAMOS to only two of the previously analyzed multi-objective Python frameworks, namely pymoo and jMetalPy. DEAP does not include built-in implementations of SMS-EMOA or MOEA/D, Platypus lacks SMS-EMOA and has seen reduced maintenance activity in recent years, and PyGMO does not include MOEA/D and it is very restrictive in terms of algorithmic configurability. Nevertheless, we include DEAP, Platypus, and PyGMO in the experiments that involve only the standard generational NSGA-II.

B. Benchmark Suites

We evaluate runtime and solution quality on widely used synthetic benchmarks: ZDT1–4 and ZDT6 [24] (two objectives; ZDT5 is excluded because it uses a binary encoding), DTLZ1–7 [25] (three objectives), and WFG1–9 [26] (two objectives). Problem dimensionalities are fixed to standard definitions: for example, DTLZ2/3/4 use $n = m + k - 1$ with $m = 3$ and $k = 10$ (thus $n = 12$). Non-standard DTLZ dimensionalities are permitted but explicitly flagged to avoid accidental comparisons against canonical settings. Unless otherwise stated, each (problem, framework) configuration is executed for 50,000 objective evaluations with 30 independent seeds. We have fixed the number of evaluations to simplify the comparisons, although it is well-known that the stopping condition to solve the ZDT and bi-objective WFG benchmarks is typically 20,000/25,000 evaluations.

C. Cross-framework Semantic Alignment

Fig. 3 summarizes the benchmark protocol and semantic-alignment workflow used to ensure cross-framework fairness and to support the “same quality, faster” claim.

To ensure comparability across frameworks, we standardize the algorithmic settings of NSGA-II: population size $N = 100$, SBX crossover probability $p_c = 1.0$ and distribution index

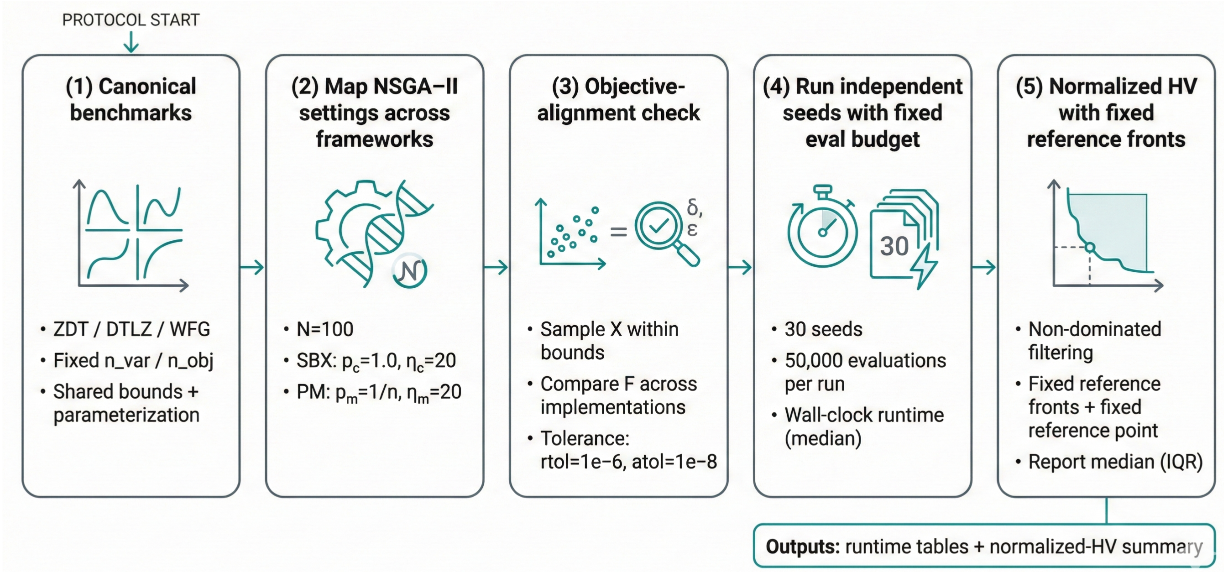

Fig. 3. Benchmark protocol and semantic alignment: map equivalent NSGA-II settings across frameworks, validate objective definitions by sampling decision vectors within bounds, run independent seeds under a fixed evaluation budget, and compute normalized hypervolume using fixed reference fronts.

$\eta_c = 20$, polynomial mutation distribution index $\eta_m = 20$ and mutation probability $p_m = 1/n$, and binary tournament selection. For each framework we map these settings to the closest available implementation. The initial population is filled with randomly initialized solutions.

For SMS-EMOA, we use the same common settings as NSGA-II. Compared with this algorithm, its main differences are the use of hypervolume contribution as density estimator, random parent selection, and a steady-state $(\mu+1)$ loop (one offspring per iteration). However, the hypervolume-contribution reference point is not defined consistently across frameworks: VAMOS and jMetalPy compute contributions in raw objective space with an adaptive reference point $r = \max(F) + 1$, while pymoo normalizes objectives and uses a fixed shifted reference point $r = \mathbf{1} + \epsilon$ (we set $\epsilon = 1$). We therefore report the cross-framework SMS-EMOA comparison with this caveat.

For MOEA/D, we align population size $N = 100$, weight vectors (uniform for two objectives; a shared $W3D\_100$ set for three objectives), neighborhood size $T = 20$, neighborhood selection probability $\delta = 0.9$, and replacement limit $\eta = 20$ (effectively unrestricted, matching pymoo's update rule). We use penalty-based boundary intersection (PBI) scalarization with $\theta = 5$ and the SBX and polynomial mutation operators with the same parameters as NSGA-II. The weight vectors for the three-objective problems are generated using the Riesz s-energy method proposed in [41], available in https://www.egr.msu.edu/coinlab/blankjul/uniform/.

This workflow follows standard practice in comparative MOEA studies (independent runs, quality indicator computation, and structured reporting) and is consistent with the experimentation methodology described in jMetalPy [7]. Our main extension is to make the *inputs* to this workflow comparable across frameworks by enforcing semantic alignment of both problem definitions and operator probabilities.

Crucially, we align *semantics* (not only parameter names) and enforce consistent benchmark definitions. For example, in **pymoo** the polynomial mutation operator exposes both an individual-level probability (`prob`) and a per-variable probability (`prob_var`); to match the standard "$p_m = 1/n$ per decision variable" setting used by jMetalPy, we set `prob`=1 and `prob_var`=1/n.

For WFG we adopt pymoo's canonical parameterization for two objectives ($k = 4$, $l = 20$ for $n = 24$) and pass parameters explicitly when a toolkit exposes different defaults. While full equivalence cannot be guaranteed due to framework-specific details (tie-breaking, boundary handling, duplicate elimination, etc.), these choices aim to make the comparison as fair as possible by removing confounders unrelated to algorithmic overhead and numerical kernels.

As an additional validity check, we sample 64 random decision vectors per problem uniformly within the specified bounds and verify that objective values match across implementations within a relative tolerance of 10^{-6} and an absolute tolerance of 10^{-8}. This guards against silent benchmark-definition drift (e.g., differing domains or parameterizations) that could invalidate hypervolume comparisons.

D. Runtime measurement

All runtimes are measured wall-clock using high-resolution timers and include algorithm overhead and objective evaluations.

For Numba-accelerated backends, we distinguish two timing policies. **Warm** timings exclude one-time JIT compilation by performing a short warmup run in the same process before starting the timer (2,000 warmup evaluations; not counted in the reported time). **Cold** timings measure the first run from a fresh process and therefore include JIT compilation overhead. Unless otherwise stated, we report warm timings to reflect steady-state throughput; cold-start costs can be reproduced with the provided scripts.

Experiments were run on an MSI Stealth 16 AI Studio A1VGG laptop with an Intel(R) Core(TM) Ultra 9 185H central processing unit (CPU) (16 cores, 22 logical processors) and 32 GB random-access memory (RAM), running Microsoft Windows 10 Home (version 25H2, build 26200). We used Python 3.12.3 and evaluated VAMOS 0.1.0 (NumPy 2.3.5 with OpenBLAS 0.3.30; Numba 0.63.1) against pymoo 0.6.1.6 and jMetalPy.

E. Quality indicators

Hypervolume (HV), denoted $\mathrm{HV}(A, r)$, measures the Lebesgue measure of the region dominated by an approximation set A with respect to a reference point r (for minimization) [42]. Because HV is scale-dependent and sensitive to the choice of r, naive implementations can produce values that are not comparable across runs (e.g., if r is expanded per run) and can be misleading when dominated solutions are included. To ensure a stable and comparable quality metric, we adopt the following protocol:

1) **Non-dominated filtering**: for every framework we compute HV on the final non-dominated set only.
2) **Fixed reference Pareto fronts**: for each benchmark problem we store a dense reference Pareto front P_{ref}. For ZDT and DTLZ (except DTLZ7) these fronts are generated analytically; for DTLZ7 and WFG we generate P_{ref} by dense sampling followed by non-dominated filtering.
3) **Fixed reference point**: we set $r = \max(P_{\text{ref}}) + \epsilon$ (component-wise) with a small ϵ, and we disallow run-dependent expansion.
4) **Normalization**: we report $\mathrm{HV}(A, r)/\mathrm{HV}(P_{\text{ref}}, r)$, yielding a dimensionless score typically in $[0, 1]$ and reducing sensitivity to objective scaling.

For WFG problems, the reference front is necessarily an approximation; we use large Pareto-set samples and non-dominated filtering, and we increase density for particularly sensitive cases (e.g., WFG2) to avoid underestimating $\mathrm{HV}(P_{\text{ref}}, r)$ and obtaining normalized HV slightly above 1.

F. VAMOS backend comparison

In the first experiment, we compare the performance of NSGA-II using the different backends of VAMOS on the ZDT, DTLZ and WFG families. For each family, we sum the execution times of solving all the problems in the family and report the median runtime of repeating this process 30 times. The results are included in Table III.

TABLE III
VAMOS BACKEND COMPARISON: MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY

Backend	ZDT	DTLZ	WFG	ZCAT	Average
Numba	**13.73**	**21.80**	**32.38**	—	**22.64**
MooCore	21.01	36.45	53.96	—	37.14
NumPy	35.14	68.86	52.68	—	52.23

The Numba backend is the fastest across all three families. MooCore is 1.5–1.7× slower than Numba, while the pure NumPy backend is 1.6–3.2× slower, confirming that per-generation interpreted Python overhead dominates the cost in the absence of JIT compilation. The gap is largest on DTLZ, where NumPy is 3.2× slower than Numba, while on WFG all three backends are closest in relative terms.

We proceed next to quantify how the array-native architecture scales when the population size increases, so we measure wall-clock runtimes of the Numba and NumPy backends on three representative problems (ZDT4, DTLZ2, WFG2) using NSGA-II with 50,000 evaluations and 5 independent seeds per configuration. Population sizes range from 50 to 800; all other settings match the main benchmark protocol.

Fig. 4 shows that the runtime gap between NumPy and Numba widens substantially as the population grows. At $N = 50$ the two backends perform similarly (and NumPy is even marginally faster on ZDT4, likely because the JIT warm-up cost is not fully amortized for small arrays), but by $N = 800$ the NumPy backend requires about 30 s on all three problems while Numba stays below 12 s—a roughly 2.5× speedup. The effect is consistent across problem families despite their different dimensionalities ($n_{\text{var}} = 10$, 12, and 24 for ZDT4, DTLZ2, and WFG2 respectively). Crucially, the Numba curve remains nearly flat across population sizes, confirming that the JIT-compiled kernels keep per-generation overhead nearly constant regardless of population size, whereas NumPy's interpreted dispatch overhead accumulates with each generation cycle. These results reinforce the practical relevance of the vectorized design for large-scale benchmarking studies that require large populations or many independent seeds under a fixed compute budget.

G. Cross-Framework Comparison

In this subsection we first compare the runtime of VAMOS against other Python frameworks using generational NSGA-II. Then, we extend the comparison to NSGA-II variants (steady-state and external archive), SMS-EMOA, and MOEA/D.

Table IV compares the runtime of generational NSGA-II across five Python frameworks. VAMOS (Numba backend) is the fastest in all three problem families. jMetalPy is the closest competitor on ZDT and DTLZ (1.8× and 1.5× slower, respectively), but its WFG runtime explodes to 13× due to per-solution overhead amplified by higher dimensionality. pymoo is consistently 2.4–2.9× slower across all families. DEAP and Platypus are substantially slower (10–37×), reflecting the overhead of per-solution object-oriented representations and interpreted inner loops.

We now compare the runtime of NSGA-II variants (steady-state and external unbounded archive), SMS-EMOA, and MOEA/D across the three frameworks that support them.The median runtimes are reported in Table V. In the steady-state variant of NSGA-II, only one offspring is produced per iteration, so the main loop executes N times more iterations than the generational variant for the same evaluation budget, introducing additional per-iteration overhead. The variant with an external unbounded archive implies that whenever a new

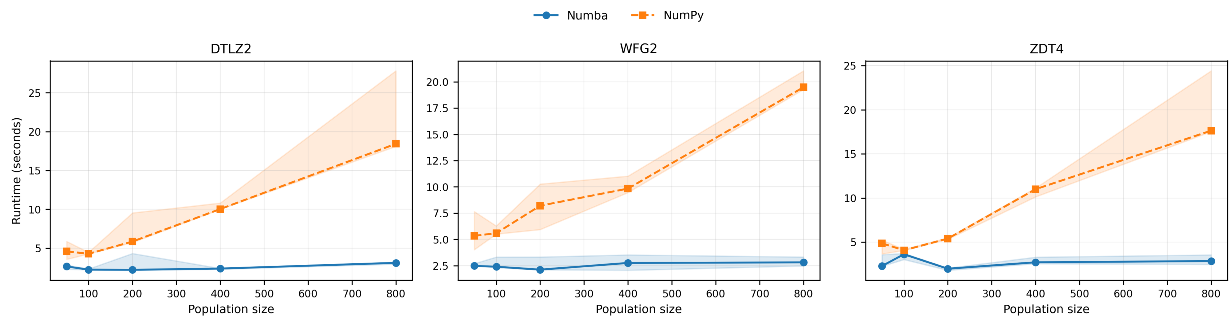

Fig. 4. Runtime scaling of VAMOS backends (Numba vs. NumPy) as a function of population size on three representative problems. Shaded regions indicate the interquartile range across 5 seeds. NumPy runtime grows steeply with population size due to increasing Python-level overhead in per-generation operations, while Numba runtime remains nearly flat, demonstrating that JIT-compiled kernels amortize loop overhead effectively on larger arrays.

TABLE IV
NSGA-II WITH STANDARD SETTINGS. MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY ACROSS ALL FRAMEWORKS.

Framework	ZDT	DTLZ	WFG	ZCAT
VAMOS	13.73	21.80	32.38	—
pymoo	39.90	60.43	77.12	—
jMetalPy	24.09	32.30	421.20	—
DEAP	163.78	226.99	1154.80	—
Platypus	203.58	270.24	1188.44	—
PyGMO	**1.00**	**1.46**	**2.59**	—

solution is evaluated, it is compared with the archive: if the new solution is non-dominated, it is inserted and any archive members it dominates are removed. This process becomes more time-consuming as the archive grows. Once the algorithm terminates, a subset of N diverse solutions is selected from the archive and returned as the final approximation set. SMS-EMOA uses the hypervolume contribution as density estimator to select the worst individual to remove from the population in the replacement step, which is a time-consuming procedure given that the computation of the hypervolume contribution grows exponentially with the number of objectives. MOEA/D uses a decomposition strategy that, a priori, should not introduce a noticeable overhead.

TABLE V
MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY FOR NSGA-II VARIANTS, SMS-EMOA, AND MOEA/D.

Algorithm	Framework	ZDT	DTLZ	WFG	ZCAT
NSGA-II (steady-state)	VAMOS	**569.64**	**963.46**	**1631.19**	—
	pymoo	5747.47	7259.88	6382.73	—
	jMetalPy	1930.43	2506.83	2756.07	—
NSGA-II (ext. archive)	VAMOS	**633.53**	**3043.39**	**253.26**	—
	pymoo	1422.71	7208.12	1515.89	—
	jMetalPy	744.93	3993.47	1736.74	—
SMS-EMOA	VAMOS	**363.02**	642.53	1342.82	—
	pymoo	1658.10	2439.60	3410.49	—
	jMetalPy	377.01	**529.49**	**1161.51**	—
MOEA/D	VAMOS	**190.11**	**292.53**	**1686.29**	—
	pymoo	1325.29	1735.46	4143.63	—
	jMetalPy	303.00	430.14	4267.52	—

VAMOS is the fastest framework for the steady-state NSGA-II, external archive, and MOEA/D configurations across all problem families. The speedup is most pronounced for the steady-state variant, where VAMOS is 3.9–10× faster than pymoo and 1.7–3.4× faster than jMetalPy. These large margins are expected: steady-state algorithms execute N iterations per generation, amplifying the per-iteration overhead of object-oriented frameworks. For the external archive variant, VAMOS is 2.2–6× faster than pymoo and 1.2–6.9× faster than jMetalPy. For MOEA/D, VAMOS is 2.5–7× faster than pymoo and 1.5–2.5× faster than jMetalPy across all families. For SMS-EMOA, VAMOS is 2.5–4.6× faster than pymoo, but jMetalPy is slightly faster than VAMOS on DTLZ and WFG (0.8–0.9×), while VAMOS retains the lead on ZDT.

H. Memory Footprint

To complement the runtime comparison, we measure peak resident-set-size (RSS) overhead during NSGA-II runs on three representative problems (ZDT1, DTLZ2, WFG4) using 50,000 evaluations and 5 seeds per framework (Fig. 6). VAMOS achieves a median peak memory of 0.28 MB, compared to 0.84 MB for pymoo (3× higher) and 0.50 MB for jMetalPy (1.8× higher). The lower memory footprint reflects the array-native representation, which stores the entire population in contiguous NumPy arrays rather than materializing individual solution objects with per-instance attribute dictionaries.

I. Convergence, Solution Quality, and Statistical Analysis

To verify that the runtime speedups do not compromise solution quality, the supplementary material provides three complementary analyses. Appendix A presents a convergence analysis: normalized hypervolume is tracked at 1,000-evaluation intervals over 50,000 evaluations on three representative problems (DTLZ2, WFG4, ZDT1), showing that VAMOS, pymoo, and jMetalPy follow nearly indistinguishable trajectories. Appendix B reports the solution quality comparison: all frameworks attain virtually identical normalized HV on ZDT, medians are tightly clustered on DTLZ, and VAMOS achieves the highest median on WFG (0.934 vs. 0.846 for pymoo). Appendix C details the statistical analysis: Wilcoxon signed-rank tests with Holm correction reveal only one significant difference out of 21 problems (ZDT6, $\Delta \approx 0.3\%$), and paired-bootstrap equivalence tests confirm that VAMOS is equivalent

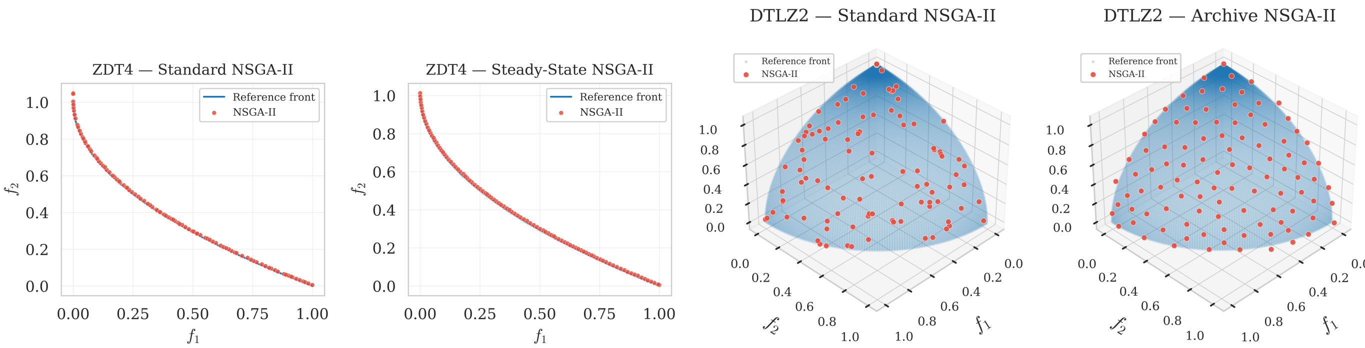

Fig. 5. Final non-dominated sets produced by NSGA-II variants on ZDT4 (left pair, 2-objective) and DTLZ2 (right pair, 3-objective), overlaid with the reference Pareto front. From left to right: (a) standard generational NSGA-II on ZDT4, (b) steady-state NSGA-II on ZDT4, (c) standard NSGA-II on DTLZ2, and (d) NSGA-II with unbounded external archive on DTLZ2. Both variants produce a more uniformly distributed approximation of the Pareto front.

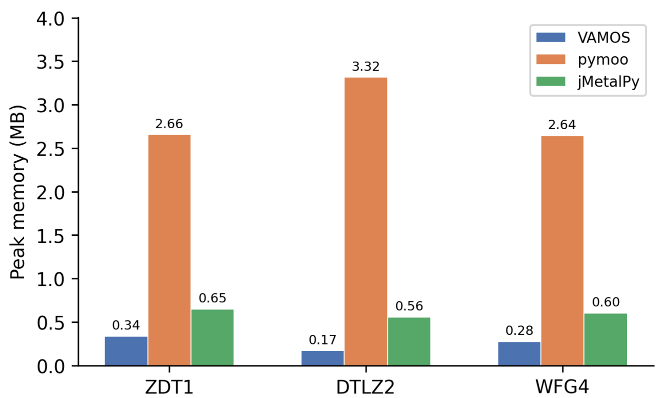

Fig. 6. Peak memory footprint (MB) during NSGA-II optimization on three representative problems (50,000 evaluations, median of 5 seeds). VAMOS consistently uses less memory than both pymoo and jMetalPy across all problem families.

to pymoo on 14/21 problems and non-inferior on 16/21. Taken together, these results confirm that VAMOS delivers faster runtimes without sacrificing convergence behavior or final solution quality.

J. Discussion

The observed speedups are primarily due to (i) the array-based representation that removes per-solution Python overhead, and (ii) JIT compilation of hot operators and utilities in the Numba backend. The scalability study (Fig. 4) confirms this mechanism: the NumPy backend's runtime grows steeply with population size, reflecting per-generation interpreted overhead, while the Numba backend remains nearly flat. The solution quality analysis in Appendix B of the supplementary material summarizes that these performance gains are not obtained at the expense of final solution quality under the fixed-reference-front normalized-HV protocol, avoiding artifacts caused by run-dependent reference points.

The cross-framework gaps are problem-family dependent: WFG problems exhibit the largest relative slowdowns for object-centric libraries, consistent with their higher dimensionality amplifying per-solution overhead. The magnitude of the speedup also varies across algorithm variants in a pattern consistent with this overhead model. Steady-state variants (Table V) show the largest relative speedups because they perform one selection–variation–survival cycle per offspring: frameworks with high per-iteration Python overhead incur that cost 50,000 times (one per evaluation) rather than 500 times (one per generation of 100 offspring). VAMOS's array-kernel dispatch amortizes this overhead more effectively, widening the gap.

Not all comparisons favor VAMOS. For SMS-EMOA on DTLZ and WFG (Table V), jMetalPy is slightly faster than VAMOS (0.8–0.9×). Unlike steady-state NSGA-II—where VAMOS's Numba-accelerated ranking and crowding-distance computation dominate the per-iteration cost—SMS-EMOA's survival selection is bottlenecked by the hypervolume contribution calculation on the worst-ranked front. Both frameworks delegate this computation to the same C library (`moocore`), so VAMOS's kernel acceleration does not yield a differential advantage. The remaining per-iteration overhead from array-based dispatch in VAMOS then becomes visible, narrowing the gap and allowing jMetalPy's leaner per-offspring loop to edge ahead on the higher-dimensional DTLZ and WFG families.

Beyond runtime, VAMOS offers a level of component-level configurability that distinguishes it from the closest competitor. pymoo provides a modular API for standard algorithm configurations, but does not natively support configuring the offspring batch size to obtain steady-state variants, attaching bounded or unbounded external archives to arbitrary algorithms, switching between multiple constraint-handling modes (feasibility rules, penalty functions, or ε-relaxation). In contrast, VAMOS exposes all of these as first-class configuration options—as demonstrated experimentally by the steady-state NSGA-II and external-archive variants (Table V), which required only parameter changes in VAMOS but custom wrappers or workarounds in other Frameworks. This configurability is complemented by the ability to swap the compute backend (NumPy, Numba, MooCore) with a single parameter, enabling users to trade compilation overhead for throughput depending on their use case.

The third design axis—accessibility—addresses a different audience than the first two. While performance and configurability primarily benefit MOEA researchers running large-scale benchmarking studies, the minimal-boilerplate API and graphical tooling target domain experts who need multi-objective optimization but lack a background in metaheuristics. The two-line `optimize` entry point, and the `make_problem` factory, eliminate the need to understand algorithm internals, operator semantics, or framework-specific abstractions. VAMOS Studio further lowers the barrier by providing a browser-based environment with domain-specific templates where users can define problems, run optimizations, and explore Pareto fronts without writing any code. As Table I shows, no other Python MOEA framework combines all three axes: array-native performance, rich configurability, and a non-verbose API with a graphical interface. We note that usability is assessed here through objective proxies (lines of code, number of required imports, and feature availability) rather than controlled user studies; a formal evaluation with domain-expert participants is left to future work.

These speedups have direct practical implications. A 4–10$\times$ reduction in per-run wall-clock time means that the same compute budget can accommodate proportionally more independent seeds, larger population sizes, or denser hyperparameter sweeps—precisely the conditions required for statistically rigorous benchmarking studies. Moreover, the scaling study (Fig. 4) shows that the advantage grows with population size, so the benefit is amplified in exactly the large-scale regimes where it matters most.

We also note that both VAMOS and pymoo obtain zero normalized hypervolume on DTLZ3 under MOEA/D with 50,000 evaluations. This is a known difficulty: DTLZ3 is a multimodal problem designed to test convergence, and MOEA/D with PBI scalarization and the evaluation budget used here is insufficient to escape local fronts consistently. This outcome is not framework-specific but reflects the interaction between algorithm configuration and problem difficulty.

A related observation concerns SMS-EMOA on DTLZ4 (see supplementary material): VAMOS attains a median normalized HV of 0.898 versus 0.671 for pymoo, yet the Wilcoxon test reports $p = 1.0$ after Holm correction. This reflects very high variance across seeds due to the deceptive density bias in DTLZ4, which causes many seeds to converge to a narrow region of the front. The high IQR makes the paired differences too noisy for the test to reject the null hypothesis despite the large median gap.

The convergence analysis (Appendix A of the supplementary material) and Pareto front visualization (Fig. 5) reinforce the solution-quality equivalence: VAMOS and pymoo follow nearly identical convergence trajectories on both ZDT1 and DTLZ2, and the final non-dominated sets from all three frameworks overlap closely with the reference fronts. These visual confirmations complement the statistical analyses and provide intuitive evidence that the runtime speedups do not degrade convergence behavior.

The memory footprint measurements (Fig. 6) further illustrate the practical advantages of the array-native design: VAMOS uses 3$\times$ less peak memory than pymoo and 1.8$\times$ less than jMetalPy on the same problems. For large-scale studies involving many concurrent runs or high-dimensional problems, this reduced memory overhead can be a meaningful practical benefit.

K. Threats to validity

Despite careful alignment of dimensions, budgets, problem definitions, and operator settings, cross-framework comparisons remain subject to several threats: (i) operator implementations may differ in subtle details (e.g., boundary handling, duplicate elimination, tie-breaking in survival), (ii) random number generation and seeding semantics vary across libraries, and (iii) library-specific overheads (data conversion, object materialization) may affect measured runtime. We mitigate major sources of unfairness by enforcing consistent benchmark bounds (e.g., ZDT4) and matching operator semantics where APIs differ (e.g., mutation probability in pymoo).

Our cross-framework comparison is deliberately scoped to Python MOEA libraries whose inner loops are predominantly executed in Python. Libraries such as PyGMO (Python bindings to the C++ PaGMO2 library) execute algorithms and benchmark functions fully in compiled code, yielding fundamentally different runtime profiles; we therefore treat them as an orthogonal C++ baseline and leave their inclusion to future work with an explicit language/runtime study design.

For hypervolume, reference fronts for problems without closed-form Pareto fronts are necessarily approximations; while we generate them densely and fix them across runs, remaining discrepancies can affect absolute normalized values, especially on WFG problems. We therefore treat normalized HV primarily as a comparative metric under a fixed protocol, and we report distributions across multiple seeds rather than relying on single-run outcomes.

VI. CONFIGURABILITY

In this section, we analyze the configurability of VAMOS by examining the parameter space of NSGA-II with real-valued encoding. Table VI lists the 35 configurable parameters organized by component (general settings, initialization, repair, crossover, and mutation). Of these, 11 are always active (e.g., population size, repair strategy, crossover type and probability, mutation type and probability factor) and 24 are conditional, i.e., they are activated only when their parent operator is selected (e.g., the distribution index η_c applies only when SBX crossover is chosen). In any single configuration, between 11 and 18 parameters are simultaneously active, depending on the selected operators.

This level of granularity contrasts sharply with competing Python frameworks, as detailed in Tables VII and VIII. In pymoo, NSGA-II exposes 20 configurable parameters with only 3 crossover and 2 mutation operators; while steady-state mode is available via `n_offsprings`, attaching an external archive requires a custom `Callback`. jMetalPy offers a broader operator catalog (29 parameters, 6 crossover and 6 mutation operators) but external archives require a custom evaluator wrapper rather than a constructor parameter. PyGMO provides even less flexibility: its NSGA-II implementation accepts only the population size, crossover probability, distribution indices, and mutation probability, with no mechanism for swapping operators or enabling algorithmic variants. Platypus, although modular in design, has limited maintenance and does not expose archive configuration or backend selection.

TABLE VI
SUMMARY OF THE CONSIDERED PARAMETER SPACE FOR NSGA-II (REAL-VALUED ENCODING). PARAMETERS IN ITALICS ARE CONDITIONAL, ACTIVE ONLY WHEN THEIR CORRESPONDING OPERATOR IS SELECTED. VALUES IN **BOLD** INDICATE THE DEFAULT PARAMETERS OF NSGA-II. THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS IS 35. ACRONYMS: LHS (LATIN HYPERCUBE SAMPLING), OBL (OPPOSITION-BASED LEARNING), SBX (SIMULATED BINARY CROSSOVER), BLX (BLEND CROSSOVER), PCX (PARENT CENTRIC CROSSOVER), UNDX (UNIMODAL NORMAL DISTRIBUTION CROSSOVER), SPX (SIMPLEX CROSSOVER), PM (POLYNOMIAL MUTATION), SUS (STOCHASTIC UNIVERSAL SAMPLING). THE STEADY-STATE SETTING CORRESPONDS TO PRODUCING ONE OFFSPRING PER ITERATION; IN THE TUNING BACKEND, THIS CASE IS INTERNALLY ENCODED AS `OFFSPRING_RATIO`=0. WHEN A BOUNDED EXTERNAL ARCHIVE IS ENABLED IN THE TUNING SPACE, ITS TYPE IS FIXED TO `SIZE_CAP` AND ITS CAPACITY IS FIXED TO THE POPULATION SIZE, SO NEITHER IS ITSELF TUNABLE

Component	Value / Strategy	Conditional Parameters
General	Pop. Size $\in [20, 200]_{\mathbb{Z}}$ (log-scale); default **100**	–
	Offspring Update $\in \{\text{steady-state}, 0.25, 0.5, 0.75, \mathbf{1.0}\}$	–
	Selection $\in$ {**Tournament**, Random, Boltzmann, Ranking, SUS}	–
	Tournament Size $\in [\mathbf{2}, 10]_{\mathbb{Z}}$	*Selection* = Tournament
	Use External Archive $\in$ {True, **False**}	–
	Archive Mode $\in$ {**bounded**, *unbounded*}	*Archive* = True
	Archive Prune Policy $\in$ {**crowding**, hv, mc_hv, knn, maxmin, ref_dirs}	*Archive* = True, *Mode* = bounded
Initialization	{**Random**, LHS, Scatter, Sobol, Halton, OBL}	–
	Scatter Base Size Factor $\in \{0.1, 0.2, 0.3, 0.5, 0.75, 1.0\}$	*Init.* = Scatter
Repair	{**clip**, reflect, random, round, wrap, midpoint}	–
Crossover	*Global:* Probability $\in [0.6, 1.0]_{\mathbb{R}}$	
	SBX	*Dist. Index* $\in [5.0, 40.0]_{\mathbb{R}}$
	BLX-α	$\alpha \in [0.0, 1.0]_{\mathbb{R}}$
	BLX-$\alpha\beta$	$\alpha \in [0.0, 1.0]_{\mathbb{R}}$, $\beta \in [0.0, 1.0]_{\mathbb{R}}$
	Arithmetic	–
	Whole Arithmetic	$\alpha \in [0.0, 1.0]_{\mathbb{R}}$
	Laplace	$a \in [-1.0, 1.0]_{\mathbb{R}}$, $b \in [0.01, 2.0]_{\mathbb{R}}$
	Fuzzy	$d \in [0.0, 2.0]_{\mathbb{R}}$
	PCX	$\sigma_\eta \in [0.01, 0.5]_{\mathbb{R}}$, $\sigma_\zeta \in [0.01, 0.5]_{\mathbb{R}}$
	UNDX	$\zeta \in [0.1, 1.0]_{\mathbb{R}}$, $\eta \in [0.1, 1.0]_{\mathbb{R}}$
	Simplex	$\epsilon \in [0.1, 1.0]_{\mathbb{R}}$
Mutation	*Global:* Prob. Factor $\in [0.25, 3.0]_{\mathbb{R}}$ ($p_m = \text{factor}/n$), Dist. Index $\in [5.0, 40.0]_{\mathbb{R}}$	
	Polynomial	(uses global dist. index)
	Linked Polynomial	(uses global dist. index)
	Non-Uniform	*Perturbation* $\in [0.05, 0.5]_{\mathbb{R}}$
	Gaussian	$\sigma \in [0.001, 0.5]_{\mathbb{R}}$
	Cauchy	$\gamma \in [0.001, 0.5]_{\mathbb{R}}$
	Uniform	*Perturbation* $\in [0.01, 0.5]_{\mathbb{R}}$
	Uniform Reset	–
	Lévy Flight	$\beta \in [0.5, 2.0]_{\mathbb{R}}$, *Scale* $\in [0.001, 0.1]_{\mathbb{R}}$
	Power Law	*Index* $\in [0.5, 5.0]_{\mathbb{R}}$

By contrast, VAMOS treats all 36 parameters as first-class configuration options accessible through a unified builder API (see supplementary material). This design enables systematic exploration of the algorithm design space, which is a feature provided by VAMOS (see next section), a capability that none of the compared frameworks currently offer out of the box.

The supplementary material contains the full parameter space of all the MOEAs included in VAMOS and an explanation of the parameters of each algorithm.

VII. ACCESSIBILITY

A key design goal of VAMOS is to lower the barrier to entry for multi-objective optimization without sacrificing the granular control required by expert researchers. To achieve this, the framework exposes a scalable API that caters to both audiences through a unified interface.

```python
from vamos import optimize

result = optimize("zdt1", algorithm="nsgaii")
```

Listing 1. Minimal NSGA-II run on ZDT1 in VAMOS.

For domain experts lacking a background in metaheuristics, VAMOS eliminates the boilerplate code typically required to set up an optimization pipeline. As shown in Listing 1, executing a standard algorithm with sensible defaults on a built-in benchmark requires only two lines of code.

Furthermore, defining custom problems does not require complex object-oriented subclassing. The `make_problem` factory converts any standard Python callable into a problem object, allowing users to define their objectives and constraints naturally, as demonstrated in Listing 2.

TABLE VII

PARAMETER SPACE FOR NSGA-II IN PYMOO (REAL-VALUED ENCODING). PARAMETERS IN ITALICS ARE CONDITIONAL, ACTIVE ONLY WHEN THEIR CORRESPONDING OPERATOR IS SELECTED. VALUES IN **BOLD** INDICATE DEFAULTS. THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS IS 20 (11 ALWAYS ACTIVE, 9 CONDITIONAL). EXTERNAL ARCHIVES ARE AVAILABLE VIA UTILITY CLASSES (`MultiObjectiveArchive`) BUT REQUIRE A CUSTOM `Callback` TO INTEGRATE WITH NSGA-II. ACRONYMS: SBX (SIMULATED BINARY CROSSOVER), PCX (PARENT CENTRIC CROSSOVER), DEX (DIFFERENTIAL EVOLUTION CROSSOVER), PM (POLYNOMIAL MUTATION), LHS (LATIN HYPERCUBE SAMPLING)

Component	**Value / Strategy**	**Conditional Parameters**
General	Pop. Size: integer; default $\mathbf{100}$	–
	Offspring Count (`n_offsprings`): integer; default = Pop. Size	–
	Selection $\in$ {**Tournament**, Random}	–
	Tournament Size (`pressure`): integer; default $\mathbf{2}$	*Selection* = Tournament
	Crowding Metric $\in$ {**cd**, pcd, ce, mnn, 2nn}	–
Initialization	{**Random**, LHS}	–
Repair	{**clip**, bounce-back, to-bound}	–
Crossover	*Global:* Probability $\in [0,1]_{\mathbb{R}}$; default $\mathbf{0.9}$	
	SBX	*Dist. Index* $\eta \in [3,30]_{\mathbb{R}}$; default $\mathbf{15}$. *Per-var. prob.* $\in [0.1,0.9]_{\mathbb{R}}$; default $\mathbf{0.5}$
	PCX	$\eta \in [0.01,0.3]_{\mathbb{R}}$, $\zeta \in [0.01,0.3]_{\mathbb{R}}$
	DEX	$CR \in [0,1]_{\mathbb{R}}$; default 0.7. F: real or `None`
Mutation	*Global:* Prob. $\in [0,1]_{\mathbb{R}}$; default $\mathbf{0.9}$. Per-var. prob.; default $\min(0.5, 1/n)$	
	PM	*Dist. Index* $\eta \in [3,30]_{\mathbb{R}}$; default $\mathbf{20}$
	Gaussian	$\sigma \in [0.01,0.25]_{\mathbb{R}}$; default 0.1

TABLE VIII

PARAMETER SPACE FOR NSGA-II IN JMETALPY (REAL-VALUED ENCODING). PARAMETERS IN ITALICS ARE CONDITIONAL, ACTIVE ONLY WHEN THEIR CORRESPONDING OPERATOR IS SELECTED. VALUES IN **BOLD** INDICATE DEFAULTS. THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS IS 29 (10 ALWAYS ACTIVE, 19 CONDITIONAL). EXTERNAL ARCHIVES CAN BE ATTACHED VIA A CUSTOM EVALUATOR (`SequentialEvaluatorWithArchive`), NOT A CONSTRUCTOR PARAMETER. ACRONYMS: SBX (SIMULATED BINARY CROSSOVER), BLX (BLEND CROSSOVER), UNDX (UNIMODAL NORMAL DISTRIBUTION CROSSOVER), DE (DIFFERENTIAL EVOLUTION), PM (POLYNOMIAL MUTATION)

Component	**Value / Strategy**	**Conditional Parameters**
General	Pop. Size: integer	–
	Offspring Pop. Size: integer; $= 1$ for steady-state	–
	Selection $\in$ {**Binary Tournament**, Tournament, Random}	–
	Tournament Size: integer ≥ 2	*Selection* = Tournament
	Use External Archive $\in$ {True, **False**}	–
	Archive Type $\in$ {NonDominated, CrowdingDist., DistanceBased}	*Archive* = True
	Archive Max Size: integer	*Archive* = True, bounded
Initialization	{**Random**, Injector}	–
Repair	{**Clamp**, Random Uniform, Reflective, Bound Swap}	–
Crossover	*Global:* Probability $\in [0,1]_{\mathbb{R}}$	
	SBX	*Dist. Index*; default $\mathbf{20}$
	BLX-α	α; default 0.5
	BLX-$\alpha\beta$	α, β; default 0.5 each
	Arithmetic	–
	UNDX	ζ; default 0.5. η; default 0.35
	DE	$CR \in [0,1]_{\mathbb{R}}$, $F \in [0,2]_{\mathbb{R}}$, $K \in [0,1]_{\mathbb{R}}$
Mutation	*Global:* Probability $\in [0,1]_{\mathbb{R}}$; conventionally $1/n$	
	Polynomial	*Dist. Index*; default $\mathbf{20}$
	Simple Random	–
	Uniform	*Perturbation*; default 0.5
	Non-Uniform	*Perturbation*, *Max Iterations*
	Lévy Flight	β; default 1.5. *Step size*; default 0.01
	Power Law	δ; default 1.0

```python
from vamos import make_problem, optimize

def my_objectives(x):
    f1 = x[0]**2 + x[1]**2
    f2 = (x[0] - 1)**2 + (x[1] - 1)**2
    return [f1, f2]

problem = make_problem(
    my_objectives, n_var=2, n_obj=2,
    bounds=[(0, 2), (0, 2)]
)
result = optimize(
    problem, algorithm="nsgaii",
    max_evaluations=50000, seed=42
)
```

Listing 2. Defining and optimizing a custom problem.

```python
from vamos import optimize
from vamos.algorithms import NSGAIIConfig

cfg = (NSGAIIConfig.builder()
       .pop_size(100)
       .crossover("sbx", prob=1.0, eta=20)
       .mutation("pm", prob="1/n", eta=20)
       .selection("tournament", pressure=4)
       .external_archive(capacity=None)
       .build())

result = optimize(
    "dtlz1", algorithm="nsgaii",
    algorithm_config=cfg,
    max_evaluations=50000, seed=42
)
```

Listing 3. Builder API for fine-grained configuration.

Conversely, for advanced users who need to conduct systematic algorithm configuration and experimentation, VAMOS provides a unified Builder API. This API exposes every configurable parameter of an algorithm without requiring different entry points or custom wrappers. Listing 3 illustrates how a researcher can precisely define the population size, configure specific crossover and mutation operators, and attach an external archive, seamlessly integrating fine-grained component control into the same execution model.

Finally, accessibility in VAMOS extends beyond algorithm execution to encompass the complete experimentation lifecycle, including hyperparameter tuning. While manual configuration via the Builder API is powerful, finding optimal algorithm configurations is often a daunting task for both novice and expert users. VAMOS addresses this by providing seamless integration with state-of-the-art Model-Based Algorithm Configuration (MBAC) tools like Optuna [18]. By exposing its entire parameter space programmatically, VAMOS allows these external tuners to efficiently automate the search for optimal configurations. Listing 1 in Appendix F of the supplementary material file illustrates this integration, showing how an advanced user can leverage Optuna to automatically tune the 35 configurable parameters of NSGA-II across multiple problem families with minimal setup effort.

VIII. Conclusion

We presented VAMOS, a high-performance Python framework for multi-objective optimization studies that addresses three practical gaps in existing Frameworks: (i) a vectorized core with pluggable compute kernels that reduces inner-loop overhead; (ii) rich component-level configurability—including interchangeable variation operators, configurable offspring batch size for steady-state variants, optional bounded or unbounded external archives, multiple constraint-handling modes—that enables researchers to explore algorithmic design choices without custom code; and (iii) an accessible API and graphical interface that lowers the entry barrier for domain experts with basic Python skills, requiring as few as two lines of code for a complete optimization run. We also release a reproducible cross-framework benchmarking pipeline with semantic alignment and fixed reference Pareto fronts for normalized hypervolume reporting. Under this protocol, VAMOS achieves competitive normalized hypervolume (see Appendix B of the supplementary material) while substantially reducing runtime (Table V). Convergence analysis (Appendix A of the supplementary material) and Pareto front visualization (Fig. 5) confirm that these speedups do not degrade convergence behavior or solution quality. The framework achieves $3\times$ lower peak memory than pymoo.

A. Future work

- **Many-objective benchmarking**: extend the experimental protocol beyond $m = 3$ with scalable indicators.
- **Distributed GPU execution**: explore integration with EvoX [12] for multi-node GPU acceleration.
- **Algorithm configuration**: extend the tuning module with interoperability with external configurators such as irace [17], and richer tuning benchmarks and reports.
- **Broader comparisons**: extend the benchmark suite to additional MOEAs and frameworks and include time-to-target and other indicators alongside normalized HV.
- **LLM-assisted experiment planning**: we are exploring an integrated assistant that maps a natural-language problem description to a validated experiment configuration, using an optional LLM provider, to further lower the barrier to entry for non-expert users.
- **User-friendliness evaluation**: conduct controlled user studies with domain experts from fields such as engineering, biology, and operations research to formally assess the usability of the API, the graphical interface, and the documentation, and to identify concrete improvements that further reduce the learning curve for non-specialist users.

Data and Code Availability

All data and code supporting the findings of this study (benchmark scripts, reference Pareto fronts, and CSV artifacts used to regenerate tables) are available in the VAMOS repository:
https://github.com/vamos-framework/vamos.

References

[1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, UK: John Wiley & Sons, 2001.

[2] C. A. C. Coello, G. B. Lamont, and D. A. V. Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY, USA: Springer, 2007.

[3] S. Bleuler, M. Laumanns, L. Thiele, and E. Zitzler, "PISA—a platform and programming language independent interface for search algorithms," in *Proc. Evolutionary Multi-Criterion Optimization (EMO)*, ser. Lecture Notes in Computer Science, vol. 2632. Springer, 2003, pp. 494–508.

[4] J. J. Durillo and A. J. Nebro, "jMetal: A Java framework for multi-objective optimization," *Advances in Engineering Software*, vol. 42, no. 10, pp. 760–771, 2011.

[5] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.

[6] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[7] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, and J. Del Ser, "jMetalPy: A Python framework for multi-objective optimization with metaheuristics," *Swarm and Evolutionary Computation*, vol. 51, p. 100598, 2019.

[8] D. Hadka, "Platypus: A free and open source Python library for multiobjective optimization," https://github.com/Project-Platypus/Platypus, accessed: 2026.

[9] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.

[10] H. Ishibuchi, L. M. Pang, and K. Shang, "A new framework of evolutionary multi-objective algorithms with an unbounded external archive," in *ECAI 2020 - 24th European Conference on Artificial Intelligence, 29 August-8 September 2020, Santiago de Compostela, Spain, August 29 - September 8, 2020 - Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020)*, ser. Frontiers in Artificial Intelligence and Applications, G. D. Giacomo, A. Catalá, B. Dilkina, M. Milano, S. Barro, A. Bugarín, and J. Lang, Eds., vol. 325. IOS Press, 2020, pp. 283–290.

[11] F. Biscani and D. Izzo, "A parallel global multiobjective framework for optimization: pagmo," *Journal of Open Source Software*, vol. 5, no. 53, p. 2338, 2020. [Online]. Available: https://doi.org/10.21105/joss.02338

[12] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, "EvoX: A distributed GPU-accelerated framework for scalable evolutionary computation," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 5, pp. 1649–1662, 2025.

[13] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 102–116, Feb. 2021.

[14] H. Zhao, X.-H. Ning, J.-Y. Li, and J. Liu, "Evolutionary multitask framework with bi-knowledge transfer for multimodal optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 393–407, Feb. 2026.

[15] H. Ye, H. Xu, and C. A. Coello Coello, "Light-EvoOPT: A lightweight evolutionary optimization framework for ultralarge-scale mixed integer linear programs," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 333–347, Feb. 2026.

[16] Z. Ma, Y.-J. Gong, H. Guo, J. Chen, Y. Ma, Z. Cao, and J. Zhang, "LLaMoCo: Instruction tuning of large language models for optimization code generation," *IEEE Transactions on Evolutionary Computation*, 2026, early access.

[17] M. L.-I. nez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle, and M. Birattari, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.

[18] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2019, pp. 2623–2631.

[19] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile Bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.

[20] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 80, 2018, pp. 1437–1446.

[21] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Krevelen, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[22] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2015.

[23] M. L.-I. nez *et al.*, "moocore: Multi-objective optimization core library," https://github.com/multi-objective/moocore, accessed: 2026.

[24] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evolutionary Computation*, vol. 8, no. 2, pp. 173–195, 2000.

[25] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable multi-objective optimization test problems," in *Proc. Congr. Evol. Comput. (CEC)*, 2002, pp. 825–830.

[26] S. Huband, P. Hingston, L. Barone, and L. While, "A review of multiobjective test problems and a scalable test problem toolkit," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 5, pp. 477–506, 2006.

[27] S. Zapotecas-Martínez, C. A. Coello Coello, H. E. Aguirre, and K. Tanaka, "Challenging test problems for multi- and many-objective optimization," *Swarm and Evolutionary Computation*, vol. 81, p. 101350, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210650223001232

[28] Q. Zhang, A. Zhou, S. Zhao, P. N. Suganthan, W. Liu, and S. Tiwari, "Multiobjective optimization test instances for the CEC 2009 special session and competition," University of Essex and Nanyang Technological University, Tech. Rep. CEC-MO09, 2008.

[29] R. Tanabe and H. Ishibuchi, "An easy-to-use real-world multi-objective optimization problem suite," *Applied Soft Computing*, vol. 89, p. 106078, 2020.

[30] S. Z. Martínez, A. García-Nájera, and A. Menchaca-Méndez, "Engineering applications of multi-objective evolutionary algorithms: A test suite of box-constrained real-world problems," *Engineering Applications of Artificial Intelligence*, vol. 122, p. 106192, 2023.

[31] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[32] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[33] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," ETH Zurich, Zurich, Switzerland, Tech. Rep. TIK-Report 103, 2001.

[34] A. Panichella, "An adaptive evolutionary algorithm based on non-Euclidean geometry for many-objective optimization," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*. ACM, 2019, pp. 595–603.

[35] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[36] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "A reference vector guided evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 5, pp. 773–791, 2016.

[37] N. Beume, B. Naujoks, and M. Emmerich, "SMS-EMOA: Multiobjective selection based on dominated hypervolume," *European Journal of Operational Research*, vol. 181, no. 3, pp. 1653–1669, 2007.

[38] E. Zitzler and S. Künzli, "Indicator-based selection in multiobjective search," in *Proc. Int. Conf. Parallel Problem Solving from Nature (PPSN)*. Springer, 2004, pp. 832–842.

[39] Streamlit Inc., "Streamlit: A faster way to build and share data apps," https://streamlit.io, 2019, accessed: 2025-01-15.

[40] J. J. Durillo, A. J. Nebro, F. Luna, and E. Alba, "On the effect of the steady-state selection scheme in multi-objective genetic algorithms," in *Evolutionary Multi-Criterion Optimization*, M. Ehrgott, C. M. Fonseca, X. Gandibleux, J.-K. Hao, and M. Sevaux, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 183–197.

[41] J. Blank, K. Deb, Y. D. Dhebar, S. Bandaru, and H. Seada, "Generating well-spaced points on a unit simplex for evolutionary many-objective optimization," *IEEE Trans. Evol. Comput.*, vol. 25, no. 1, pp. 48–60, 2021.

[42] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, pp. 117–132, 2003.